



Kubernetes - Pod Troubleshooting

Check → Investigate → Analyze → Move to Next Step

Follow this flow step by step until you find the root cause.

Check Pod Status

Find the current state of pods in the namespace

Command

```
> kubectl get pods -n <namespace>
```

Example Output

NAME	READY	STATUS	RESTARTS
payment-api	1/1	Running	0
order-api	0/1	CrashLoopBackOff	15
user-api	0/1	Pending	0

What to Look For

- Pod status (Running, Pending, CrashLoopBackOff, OOMKilled, etc.)
- READY column (containers ready or not)
- High restart count
- Which pods are affected



This gives you a quick overview of the cluster pod health.

Describe & Check Events

Get detailed information and find the exact reason

Command

```
> kubectl describe pod <pod-name> -n <namespace>
```

Example Output (Events Section)

Events:			
Type	Reason	Age	Message
Warning	FailedScheduling	2m	0/3 nodes are available: insufficient cpu
Warning	BackOff	1m	Back-off restarting failed container

What to Look For

- Events section (most important)
- Scheduling errors
- Image pull errors
- Probe failures
- Resource issues (CPU/Memory)
- Node related issues
- PVC/Storage issues



Events give the exact reason why the pod is not running properly.

Check Logs

See application logs to find actual error

Command

```
> kubectl logs <pod-name> -n <namespace>
```

If the container is restarting, check previous logs:

```
> kubectl logs <pod-name> -n <namespace> --previous
```

Example Output

```
Exception in thread "main"
java.lang.RuntimeException:
Database connection failed
    at com.app.Main.main(Main.java:20)
Caused by: java.net.ConnectException:
Connection refused
```

What to Look For

- Application errors
- Missing configuration / environment variables
- Dependency (DB/API) connection issues
- Any exceptions or stack traces
- Logs from previous container (if crash)



Logs show the real application issue.

Check Resource Usage

Verify if the issue is due to high CPU/Memory or node resources

Commands

```
> kubectl top pod <pod-name> -n <namespace>
> kubectl top nodes
> kubectl describe node <node-name>
> kubectl get pvc -n <namespace>
```

Example Output (Pod Resource)

NAME	CPU(cores)	MEMORY(bytes)
order-api	950m	1.2Gi

Example Output (Node Resource)

NAME	CPU(cores)	MEMORY(bytes)
aks-node-1	3800m	7.5Gi

What to Look For

- CPU/Memory usage is high
- Nodes are out of resources
- Memory limits are too low (OOMKilled)
- PVC in Pending (storage issue)
- Node pressure (DiskPressure, MemoryPressure)
- Insufficient resources for scheduling



Helps confirm if the problem is resource related.



Kubernetes - Pod Status

Pod status tells you the current state of the Pod.

Check using: `kubectl get pods -n <namespace>`

STATUS	MEANING	EXAMPLE
Running	Pod is running and at least one container is running.	payment-api 1/1 Running
Pending	Pod is created but not scheduled on any node yet.	user-api 0/1 Pending
Succeeded	Pod completed its work (successfully). (Common for Job/CronJob)	batch-job 0/1 Succeeded
Failed	Pod failed to complete its work.	backup-job 0/1 Failed
CrashLoopBackOff	Container is repeatedly crashing and restarting.	order-api 0/1 CrashLoopBackOff
OOMKilled	Container was killed due to Out Of Memory.	service-api 0/1 OOMKilled
ImagePullBackOff	Failed to pull container image.	web-api 0/1 ImagePullBackOff
ErrImagePull	Error while pulling container image.	web-api 0/1 ErrImagePull
ContainerCreating	Container is being created (e.g., image pulling, volume mounting).	db-api 0/1 ContainerCreating
Terminating	Pod is in the process of deletion.	cache-api 1/1 Terminating
Unknown	Status of the pod cannot be determined.	api 0/1 Unknown



Kubernetes - Common Pod Issues

These are the most common pod issues in production and their possible causes.

Check details using: `kubectl describe pod <pod-name> -n <ns>`

ISSUE (STATUS)	COMMON CAUSES
Pending	- Insufficient CPU/Memory on nodes - NodeSelector / Taints / Tolerations mismatch - PVC not bound - Affinity / Anti-affinity rules - No available nodes
CrashLoopBackOff	- Application crash (check logs) - Wrong configuration / missing environment variable - Dependency (DB/API) not available - Liveness probe failure - Resource limits too low
OOMKilled	- Memory limit too low - Memory leak in application - Incorrect JVM/heap settings - High traffic/load
ImagePullBackOff	- Wrong image name or tag - Image not found in registry - ACR/Docker Hub authentication issue - Image pull secret missing - Network/DNS issue (cannot reach registry)
ContainerCreating	- Image pull delay - CNI/Network issue - PVC mounting issue - Init container failure - Node resource issue (slow image download)
Running (But App Not Working)	- Readiness probe failing - Service/Ingress issue - Application error (check logs) - Database/External API connectivity issue - Misconfiguration